Mini-projet: Microservices

1. Description du projet

Dans le cadre de ce projet, les étudiants doivent concevoir et développer une application basée sur une architecture microservices en utilisant uniquement Node.js.

L'application devra être composée d'au moins trois microservices indépendants, d'une API Gateway, d'un client ou d'une interface de test, d'un broker Kafka et de bases de données séparées pour chaque microservice.

Le projet doit être réalisé en binôme ou en trinôme, avec un suivi obligatoire du travail sur GitHub.

2. Objectifs du projet

L'objectif de ce projet est de concevoir, développer et présenter une application fonctionnelle basée sur une architecture microservices.

L'application doit démontrer une séparation claire des responsabilités entre les microservices, ainsi qu'une communication correcte entre les différents composants.

3. Architecture attendue

L'application doit respecter l'architecture suivante :

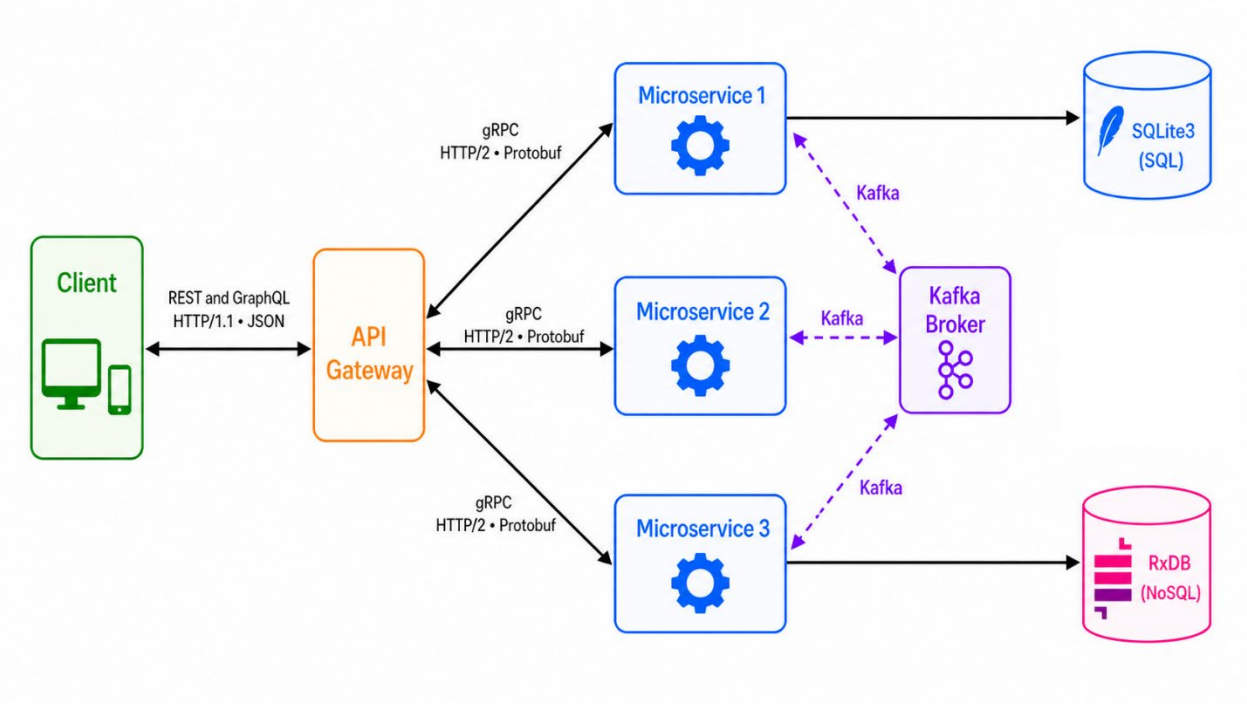


Figure 1. Architecture microservices du projet

4. Travail demandé

4.1 Conception de l'application

Les étudiants doivent concevoir une application basée sur au moins trois microservices.

Chaque microservice doit avoir un rôle clair et indépendant. L'application peut être une application de vente en ligne, une application de réservation, une application de gestion, une application IoT, une application de cybersécurité ou toute autre application qui met en évidence les interactions entre les microservices.

Le sujet choisi doit être suffisamment riche pour justifier l'utilisation de REST, GraphQL, gRPC et Kafka.

4.2 API Gateway

L'API Gateway doit jouer le rôle de point d'entrée principal de l'application.

Elle doit permettre au client d'interagir avec le système à travers :

- des endpoints REST ;
- des requêtes GraphQL.

L'API Gateway doit ensuite communiquer avec les microservices via gRPC.

Elle ne doit pas contenir toute la logique métier. La logique principale doit rester dans les microservices.

4.3 Microservices

Chaque microservice doit :

- être développé en Node.js ;
- avoir une responsabilité claire ;
- exposer une interface gRPC ;
- posséder sa propre base de données ;
- être indépendant des autres microservices ;
- communiquer avec les autres microservices via Kafka lorsque c'est nécessaire ;
- gérer correctement les erreurs simples.

Les microservices doivent être conçus comme des composants autonomes.

4.4 Communication gRPC

La communication gRPC est l'élément technique le plus important du projet.

Les étudiants doivent fournir :

- des fichiers .proto bien définis ;
- des services gRPC fonctionnels ;
- des messages Protobuf adaptés ;
- des appels gRPC entre l'API Gateway et les microservices ;
- une séparation claire entre le contrat d'interface et l'implémentation.

La qualité de l'implémentation gRPC sera fortement prise en compte dans l'évaluation.

4.5 REST

L'API Gateway doit exposer une interface REST.

REST doit être utilisé pour des opérations classiques telles que :

- création ;
- consultation ;
- modification ;
- suppression ;
- recherche simple.

Les endpoints REST doivent être clairs, cohérents et testables.

4.6 GraphQL

L'API Gateway doit exposer une interface GraphQL.

GraphQL doit être utilisé pour des requêtes flexibles, permettant au client de demander précisément les champs nécessaires.

Les étudiants doivent fournir un schéma GraphQL clair et des requêtes démontrables.

4.7 Kafka

Kafka doit être utilisé pour la communication asynchrone entre les microservices.

Il doit être utilisé pour des événements métier pertinents. Kafka ne doit pas être utilisé artificiellement uniquement pour satisfaire une contrainte technique.

Les étudiants doivent décrire :

- les topics Kafka utilisés ;
- les microservices producteurs ;
- les microservices consommateurs ;
- le contenu général des messages échangés ;

- le scénario métier déclenchant les événements.

Kafka doit servir à découpler les microservices et à faciliter la communication événementielle.

4.8 Bases de données

Chaque microservice doit disposer de sa propre base de données.

Les bases de données autorisées sont uniquement :

- SQLite3 pour les bases de données SQL ;
- RxDB pour les bases de données NoSQL.

Au moins deux microservices doivent être associés à une base de données indépendante.

5. Collaboration et organisation du travail

Le projet doit être réalisé en binôme ou en trinôme.

Un trinôme est accepté uniquement si le projet est plus complet, mieux structuré et plus avancé techniquement qu'un projet réalisé en binôme.

La collaboration doit se faire à travers GitHub.

Les étudiants doivent créer **une organisation GitHub** ou un repository bien structuré avec tous les membres du groupe.

Les commits seront vérifiés.

Les éléments suivants seront pris en compte :

- régularité des commits ;
- participation équilibrée des membres ;
- messages de commits clairs ;
- utilisation de branches ;
- organisation du code ;
- pourcentage de contribution de chaque membre ;
- cohérence entre les commits et les tâches réalisées.

Le pourcentage de contribution GitHub sera utilisé comme KPI de collaboration.

Une contribution très déséquilibrée peut impacter la note individuelle.

6. Livraison attendue

Les livrables obligatoires sont :

1. Code source complet sur GitHub.

2. Documentation technique du projet.
3. Fichier README clair.
4. Schéma d'architecture.
5. Fichiers .proto.
6. Description des endpoints REST.
7. Description du schéma GraphQL.
8. Description des topics Kafka.
9. Description des bases de données utilisées.
10. Instructions d'installation et d'exécution.

La documentation doit permettre à une personne externe de comprendre, installer et exécuter le projet.

7. Barème d'évaluation

La note finale est sur 20 points.

A. Partie technique principale — 16 points

1. gRPC — 5 points

Critère le plus important du projet.

- Définition correcte des fichiers .proto
- Implémentation correcte des services gRPC
- Communication fonctionnelle entre API Gateway et microservices
- Utilisation correcte de Protobuf et HTTP/2
- Gestion des erreurs et structuration propre du code gRPC
- Cohérence entre les contrats gRPC et la logique métier

2. REST — 3 points

- Endpoints REST fonctionnels et bien organisés
- Opérations REST adaptées au scénario métier
- Intégration correcte de REST dans l'API Gateway
- Tests ou exemples d'utilisation

3. GraphQL — 3 points

- Schéma GraphQL clair et exploitable

- Requêtes GraphQL fonctionnelles
- Intégration correcte de GraphQL dans l'API Gateway
- Justification de l'utilisation de GraphQL

4. Kafka — 3 points

- Définition claire des topics Kafka
- Producteurs Kafka fonctionnels
- Consommateurs Kafka fonctionnels
- Pertinence des événements métier utilisés
- Documentation des messages échangés

5. Fonctionnalités et qualité globale de l'application — 2 points

- Application fonctionnelle de bout en bout
- Scénarios métier clairs et démontrables
- Qualité du code et organisation du projet
- Documentation technique claire

B. Originalité, collaboration et éléments optionnels — 4 points

Cette partie valorise la qualité générale du projet au-delà de l'implémentation technique obligatoire.

Elle prend en compte :

- l'originalité du sujet ;
- la valeur ajoutée de l'application ;
- la qualité de la collaboration GitHub ;
- l'équilibre des contributions entre les membres ;
- l'organisation du travail ;
- l'utilisation éventuelle de la conteneurisation ;
- l'utilisation éventuelle du cloud.

Les commits GitHub seront vérifiés. Le pourcentage de contribution de chaque membre sera pris en compte.

La conteneurisation avec Docker/Docker Compose et le déploiement cloud sont optionnels, mais peuvent améliorer la note dans cette partie.

Idées de sujets

1. Marketplace e-commerce avancée
2. Plateforme de réservation intelligente
3. Système de livraison en temps réel
4. Plateforme de formation en ligne
5. Système IoT de supervision
6. Plateforme de détection d'incidents cybersécurité
7. Application bancaire simplifiée
8. Gestion intelligente d'un hôpital ou d'une clinique
9. Gestion de flotte de véhicules
10. Plateforme de streaming vidéo simplifiée